```
import gensim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

```
!pip install gensim

print("gensim installed successfully.")
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 53.8 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
gensim installed successfully.
```

```
import gensim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

```
import gensim.downloader as api

# 1. Download a pre-trained word embedding model
print("Downloading word2vec-google-news-300 model...")
model = api.load('word2vec-google-news-300')
print("Model downloaded successfully.")

# 2. Print the total vocabulary size of the loaded model
vocabulary_size = len(model.key_to_index)
print(f"\nTotal vocabulary size: {vocabulary_size}")

# 3. Select a common word and display its high-dimensional vector representation
example_word = 'computer'
if example_word in model:
    print(f"\nVector for '{example_word}':")
    print(model[example_word])
else:
    print(f"\n'{example_word}' not found in the vocabulary.")
```

```
Downloading word2vec-google-news-300 model...
[==================================================] 100.0% 1662.8/1662.8MB downloaded
Model downloaded successfully.

Total vocabulary size: 3000000

Vector for 'computer':
[ 1.07421875e-01 -2.01171875e-01  1.23046875e-01  2.11914062e-01
 -9.13085938e-02  2.16796875e-01 -1.31835938e-01  8.30078125e-02
  2.02148438e-01  4.78515625e-02  3.66210938e-02 -2.45361328e-02
  2.39257812e-02 -1.60156250e-01 -2.61230469e-02  9.71679688e-02
 -6.34765625e-02  1.84570312e-01  1.70898438e-01 -1.63085938e-01
 -1.09375000e-01  1.49414062e-01 -4.65393066e-04  9.61914062e-02
  1.68945312e-01  2.60925293e-03  8.93554688e-02  6.49414062e-02
  3.56445312e-02 -6.93359375e-02 -1.46484375e-01 -1.21093750e-01
 -2.27539062e-01  2.45361328e-02 -1.24511719e-01 -3.18359375e-01
 -2.20703125e-01  1.30859375e-01  3.66210938e-02 -3.63769531e-02
 -1.13281250e-01  1.95312500e-01  9.76562500e-02  1.26953125e-01
  6.59179688e-02  6.93359375e-02  1.02539062e-02  1.75781250e-01
 -1.68945312e-01  1.21307373e-03 -2.98828125e-01 -1.15234375e-01
  5.66406250e-02 -1.77734375e-01 -2.08984375e-01  1.76757812e-01
```

```
  2.38037109e-02 -2.57812500e-01 -4.46777344e-02  1.88476562e-01
  5.51757812e-02  5.02929688e-02 -1.06933594e-01  1.89453125e-01
 -1.16210938e-01  8.49609375e-02 -1.71875000e-01  2.45117188e-01
 -1.73828125e-01 -8.30078125e-03  4.56542969e-02 -1.61132812e-02
  1.86523438e-01 -6.05468750e-02 -4.17480469e-02  1.82617188e-01
  2.20703125e-01 -1.22558594e-01 -2.55126953e-02 -3.08593750e-01
  9.13085938e-02  1.60156250e-01  1.70898438e-01  1.19628906e-01
  7.08007812e-02 -2.64892578e-02 -3.08837891e-02  4.06250000e-01
 -1.01562500e-01  5.71289062e-02 -7.26318359e-03 -9.17968750e-02
 -1.50390625e-01 -2.55859375e-01  2.16796875e-01 -3.63769531e-02
  2.24609375e-01  8.00781250e-02  1.56250000e-01  5.27343750e-02
  1.50390625e-01 -1.14746094e-01 -8.64257812e-02  1.19140625e-01
 -7.17773438e-02  2.73437500e-01 -1.64062500e-01  7.29370117e-03
  4.21875000e-01 -1.12792969e-01 -1.35742188e-01 -1.31835938e-01
 -1.37695312e-01 -7.66601562e-02  6.25000000e-02  4.98046875e-02
 -1.91406250e-01 -6.03027344e-02  2.27539062e-01  5.88378906e-02
 -3.24218750e-01  5.41992188e-02 -1.35742188e-01  8.17871094e-03
 -5.24902344e-02 -1.74713135e-03 -9.81445312e-02 -2.86865234e-02
  3.61328125e-02  2.15820312e-01  5.98144531e-02 -3.08593750e-01
 -2.27539062e-01  2.61718750e-01  9.86328125e-02 -5.07812500e-02
  1.78222656e-02  1.31835938e-01 -5.35156250e-01 -1.81640625e-01
  1.38671875e-01 -3.10546875e-01 -9.71679688e-02  1.31835938e-01
 -1.16210938e-01  7.03125000e-02  2.85156250e-01  3.51562500e-02
 -1.01562500e-01 -3.75976562e-02  1.41601562e-01  1.42578125e-01
 -5.68847656e-02  2.65625000e-01 -2.09960938e-01  9.64355469e-03
 -6.68945312e-02 -4.83398438e-02 -6.10351562e-02  2.45117188e-01
 -9.66796875e-02  1.78222656e-02 -1.27929688e-01 -4.78515625e-02
 -7.26318359e-03  1.79687500e-01  2.78320312e-02 -2.10937500e-01
 -1.43554688e-01 -1.27929688e-01  1.73339844e-02 -3.60107422e-03
 -2.04101562e-01  3.63159180e-03 -1.19628906e-01 -6.15234375e-02
  5.93261719e-02 -3.23486328e-03 -1.70898438e-01 -3.14941406e-02
 -8.88671875e-02 -2.89062500e-01  3.44238281e-02 -1.87500000e-01
  2.94921875e-01  1.58203125e-01 -1.19628906e-01  7.61718750e-02
  6.39648438e-02 -4.68750000e-02 -6.83593750e-02  1.21459961e-02
 -1.44531250e-01  4.54101562e-02  3.68652344e-02  3.88671875e-01
  1.45507812e-01 -2.55859375e-01 -4.46777344e-02 -1.33789062e-01
```

```
selected_words = [
    'cat', 'dog', 'lion', 'tiger', 'elephant',
    'london', 'paris', 'tokyo', 'new_york', 'sydney',
    'computer', 'phone', 'keyboard', 'mouse', 'monitor',
    'joy', 'sadness', 'anger', 'fear', 'surprise',
    'doctor', 'teacher', 'engineer', 'artist', 'scientist',
    'mountain', 'river', 'forest', 'ocean', 'desert'
]

word_vectors = []
valid_words = []

print("Extracting word vectors...")
for word in selected_words:
    if word in model:
        word_vectors.append(model[word])
        valid_words.append(word)
    else:
        print(f"Warning: '{word}' not found in model vocabulary and will be skipped.")

word_vectors_array = np.array(word_vectors)

print(f"\nSuccessfully extracted vectors for {len(valid_words)} words.")
print(f"Shape of word_vectors_array: {word_vectors_array.shape}")
```

```
Extracting word vectors...
Warning: 'new_york' not found in model vocabulary and will be skipped.

Successfully extracted vectors for 29 words.
Shape of word_vectors_array: (29, 300)
```

```
from sklearn.manifold import TSNE

# 1. Instantiate a TSNE object with specified parameters
# n_components=2 for 2D visualization
# random_state for reproducibility
# perplexity (e.g., 30) is often a good starting point for datasets between 50 and 5000 samples
# n_iter=2500 for sufficient optimization, especially with smaller datasets
tsne = TSNE(n_components=2, random_state=42, perplexity=10, n_iter=2500)

# 2. Apply fit_transform to the word_vectors_array
# 3. Store the resulting 2D coordinates
print("Applying t-SNE dimensionality reduction...")
```

```
tsne_2d_embeddings = tsne.fit_transform(word_vectors_array)

print(f"t-SNE dimensionality reduction complete. Shape of 2D embeddings: {tsne_2d_embeddings.shape}")
```

```
Applying t-SNE dimensionality reduction...
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'n_iter' was renamed to 'max_iter' in ve
  warnings.warn(
t-SNE dimensionality reduction complete. Shape of 2D embeddings: (29, 2)
```

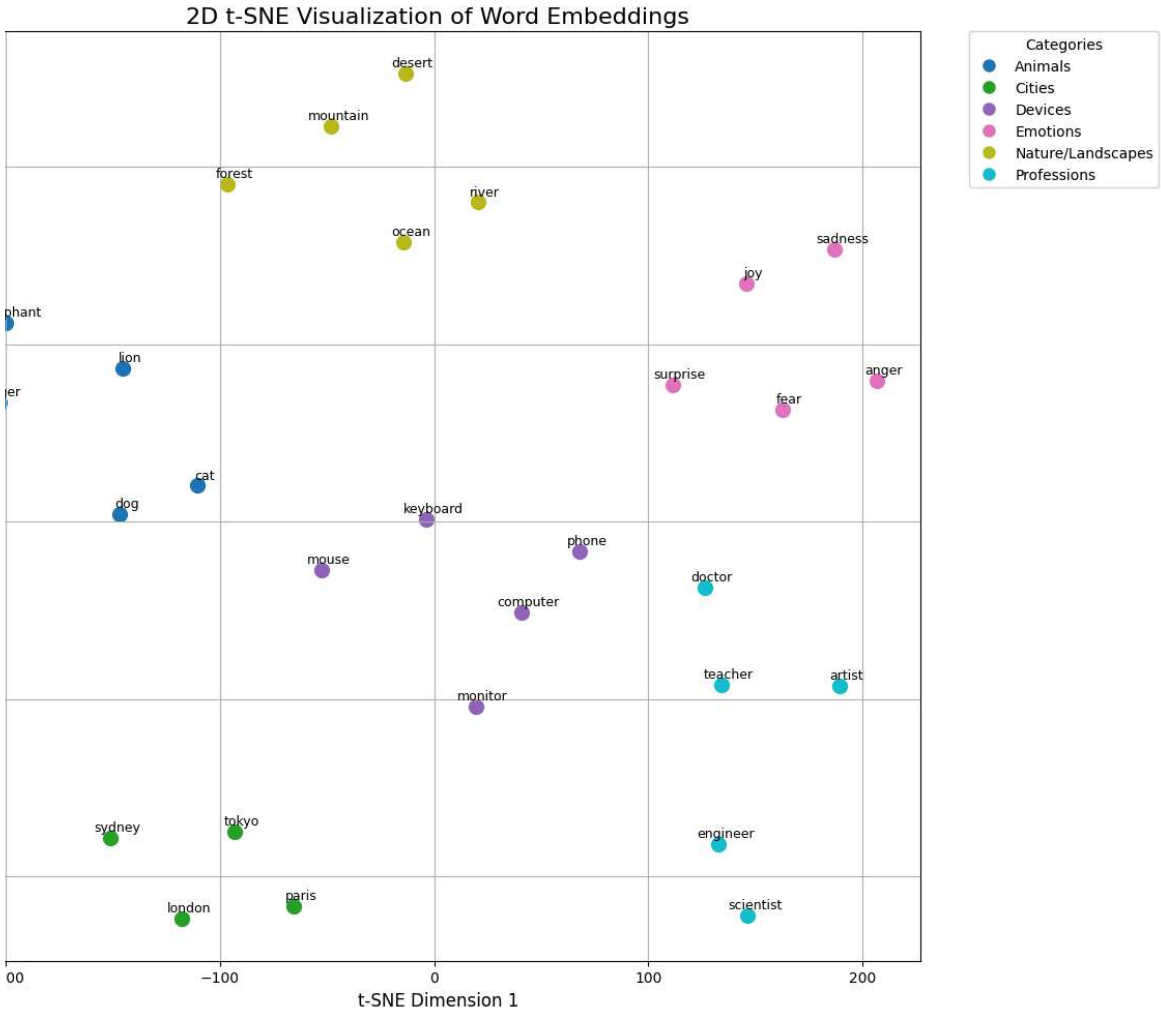
```
from sklearn.manifold import TSNE

# 1. Instantiate a TSNE object with specified parameters
# n_components=2 for 2D visualization
# random_state for reproducibility
# perplexity (e.g., 30) is often a good starting point for datasets between 50 and 5000 samples
# max_iter=2500 for sufficient optimization, especially with smaller datasets (formerly n_iter)
tsne = TSNE(n_components=2, random_state=42, perplexity=10, max_iter=2500)

# 2. Apply fit_transform to the word_vectors_array
# 3. Store the resulting 2D coordinates
print("Applying t-SNE dimensionality reduction...")
tsne_2d_embeddings = tsne.fit_transform(word_vectors_array)

print(f"t-SNE dimensionality reduction complete. Shape of 2D embeddings: {tsne_2d_embeddings.shape}")
```

```
Applying t-SNE dimensionality reduction...
t-SNE dimensionality reduction complete. Shape of 2D embeddings: (29, 2)
```

```
import matplotlib.pyplot as plt

# 1. Define a dictionary to map each word in valid_words to its semantic category
word_categories = {
    'cat': 'Animals', 'dog': 'Animals', 'lion': 'Animals', 'tiger': 'Animals', 'elephant': 'Animals',
    'london': 'Cities', 'paris': 'Cities', 'tokyo': 'Cities', 'sydney': 'Cities',
    'computer': 'Devices', 'phone': 'Devices', 'keyboard': 'Devices', 'mouse': 'Devices', 'monitor': 'Devices',
    'joy': 'Emotions', 'sadness': 'Emotions', 'anger': 'Emotions', 'fear': 'Emotions', 'surprise': 'Emotions',
    'doctor': 'Professions', 'teacher': 'Professions', 'engineer': 'Professions', 'artist': 'Professions', 'scientist': 'Profes
    'mountain': 'Nature/Landscapes', 'river': 'Nature/Landscapes', 'forest': 'Nature/Landscapes', 'ocean': 'Nature/Landscapes',
}

# Get unique categories for plotting
categories = sorted(list(set(word_categories.values())))

# Create a color map for categories
colors = plt.cm.get_cmap('tab10', len(categories))
category_to_color = {category: colors(i) for i, category in enumerate(categories)}

# 2. Create a scatter plot of the 2D word embeddings
plt.figure(figsize=(15, 10))

# Loop through each word and its embedding to plot and annotate
for i, word in enumerate(valid_words):
    x, y = tsne_2d_embeddings[i, 0], tsne_2d_embeddings[i, 1]
    category = word_categories.get(word, 'Unknown') # Handle cases where a word might not be in the map
    color = category_to_color.get(category, 'gray')

    # Plot the point
    plt.scatter(x, y, color=color, s=100, label=category if word == next(iter(word_categories)) or category not in plt.gca().ge

    # 3. Annotate each data point with its corresponding word label
    plt.annotate(word, (x, y), textcoords="offset points", xytext=(5,5), ha='center', fontsize=9)

# Create custom legend handles to avoid duplicate labels for categories
handles = []
labels = []
for category in categories:
    handles.append(plt.Line2D([0], [0], marker='o', color='w', label=category,
                              markerfacecolor=category_to_color[category], markersize=10))
    labels.append(category)

# 4. Add a title to the plot
plt.title('2D t-SNE Visualization of Word Embeddings', fontsize=16)

# 5. Add labels for the x-axis and y-axis
plt.xlabel('t-SNE Dimension 1', fontsize=12)
plt.ylabel('t-SNE Dimension 2', fontsize=12)
```

```
# 6. Include a legend to distinguish between the different semantic categories
plt.legend(handles=handles, labels=labels, title='Categories', bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0.)

# Adjust plot layout to make space for the legend
plt.tight_layout(rect=[0, 0, 0.85, 1])

# 7. Display the plot
plt.grid(True)
plt.show()

print("t-SNE visualization plot generated successfully.")
```

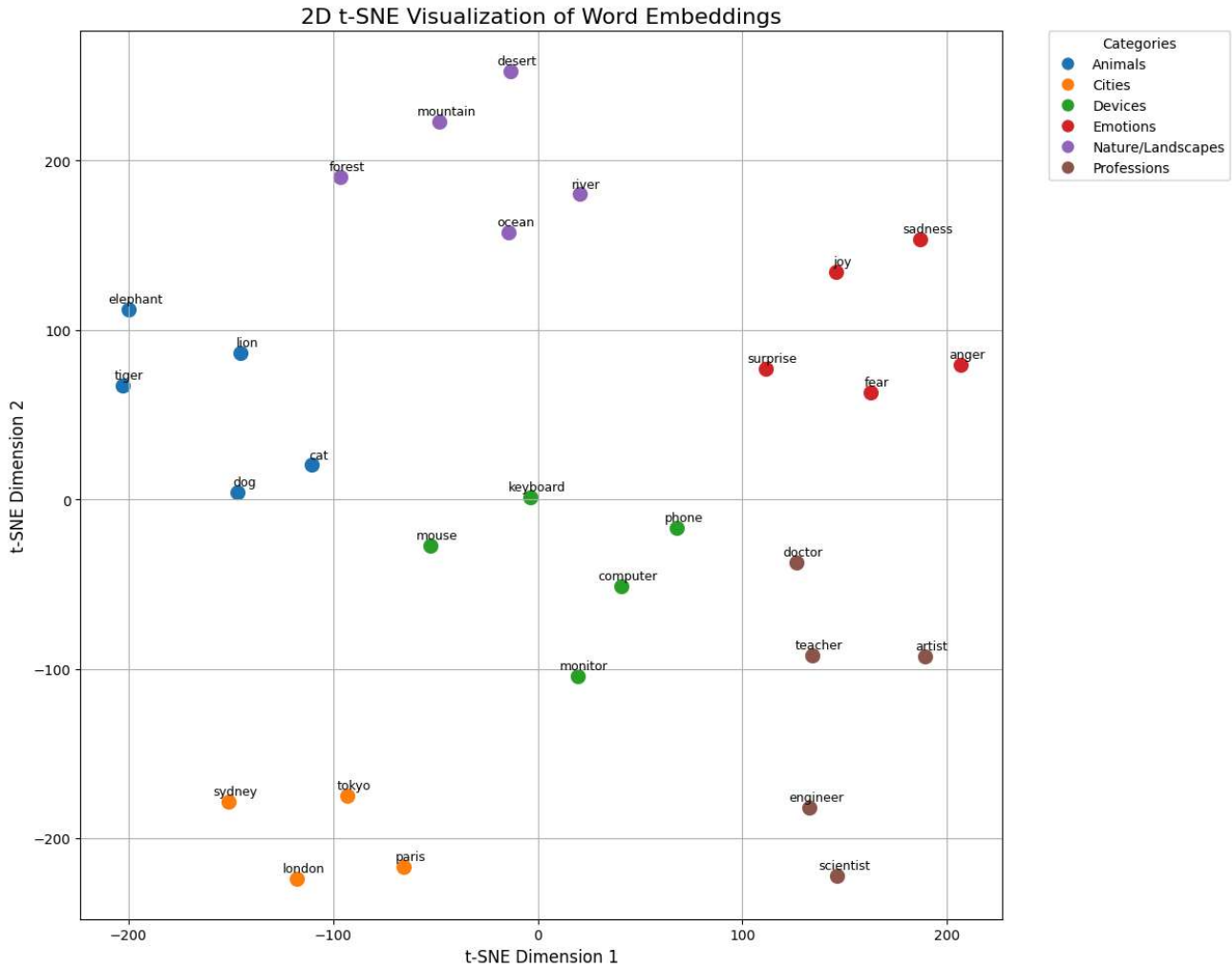
```
input-3725128108.py:17: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and wil
t.cm.get_cmap('tab10', len(categories))
```




```
zation plot generated successfully.
```

```
import matplotlib.pyplot as plt

# 1. Define a dictionary to map each word in valid_words to its semantic category
word_categories = {
    'cat': 'Animals', 'dog': 'Animals', 'lion': 'Animals', 'tiger': 'Animals', 'elephant': 'Animals',
    'london': 'Cities', 'paris': 'Cities', 'tokyo': 'Cities', 'sydney': 'Cities',
```

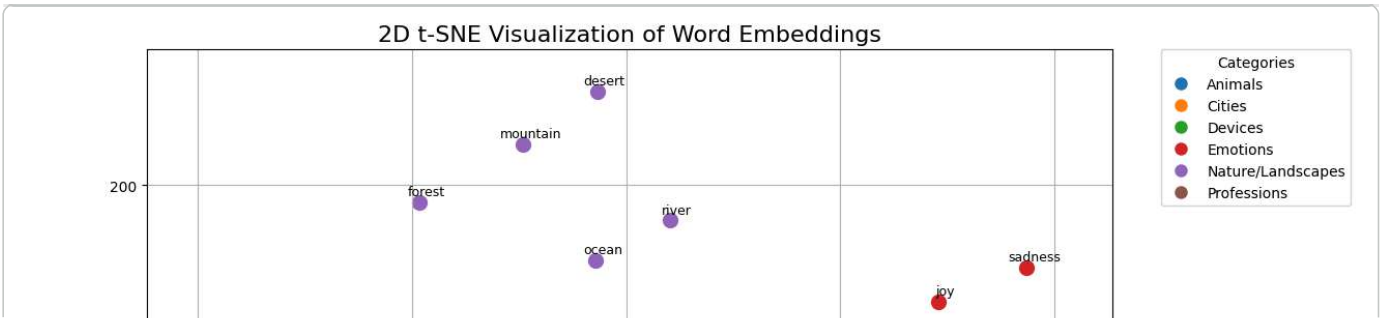
```
    'computer': 'Devices', 'phone': 'Devices', 'keyboard': 'Devices', 'mouse': 'Devices', 'monitor': 'Devices',
    'joy': 'Emotions', 'sadness': 'Emotions', 'anger': 'Emotions', 'fear': 'Emotions', 'surprise': 'Emotions',
    'doctor': 'Professions', 'teacher': 'Professions', 'engineer': 'Professions', 'artist': 'Professions', 'scientist': 'Profes
    'mountain': 'Nature/Landscapes', 'river': 'Nature/Landscapes', 'forest': 'Nature/Landscapes', 'ocean': 'Nature/Landscapes',
}

# Get unique categories for plotting
categories = sorted(list(set(word_categories.values())))

# Create a color map for categories
colors = plt.colormaps.get_cmap('tab10') # Corrected: Removed len(categories) as second argument
category_to_color = {category: colors(i) for i, category in enumerate(categories)}

# 2. Create a scatter plot of the 2D word embeddings
plt.figure(figsize=(15, 10))

# Loop through each word and its embedding to plot and annotate
for i, word in enumerate(valid_words):
    x, y = tsne_2d_embeddings[i, 0], tsne_2d_embeddings[i, 1]
    category = word_categories.get(word, 'Unknown') # Handle cases where a word might not be in the map
    color = category_to_color.get(category, 'gray')

    # Plot the point
    plt.scatter(x, y, color=color, s=100, label=category if word == next(iter(word_categories)) or category not in plt.gca().ge

    # 3. Annotate each data point with its corresponding word label
    plt.annotate(word, (x, y), textcoords="offset points", xytext=(5,5), ha='center', fontsize=9)

# Create custom legend handles to avoid duplicate labels for categories
handles = []
labels = []
for category in categories:
    handles.append(plt.Line2D([0], [0], marker='o', color='w', label=category,
                              markerfacecolor=category_to_color[category], markersize=10))
    labels.append(category)

# 4. Add a title to the plot
plt.title('2D t-SNE Visualization of Word Embeddings', fontsize=16)

# 5. Add labels for the x-axis and y-axis
plt.xlabel('t-SNE Dimension 1', fontsize=12)
plt.ylabel('t-SNE Dimension 2', fontsize=12)

# 6. Include a legend to distinguish between the different semantic categories
plt.legend(handles=handles, labels=labels, title='Categories', bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0.)

# Adjust plot layout to make space for the legend
plt.tight_layout(rect=[0, 0, 0.85, 1])

# 7. Display the plot
plt.grid(True)
plt.show()

print("t-SNE visualization plot generated successfully.")
```




```
t-SNE visualization plot generated successfully.
```

```
import matplotlib.pyplot as plt

# 1. Define a dictionary to map each word in valid_words to its semantic category
word_categories = {
    'cat': 'Animals', 'dog': 'Animals', 'lion': 'Animals', 'tiger': 'Animals', 'elephant': 'Animals',
    'london': 'Cities', 'paris': 'Cities', 'tokyo': 'Cities', 'sydney': 'Cities',
    'computer': 'Devices', 'phone': 'Devices', 'keyboard': 'Devices', 'mouse': 'Devices', 'monitor': 'Devices',
    'joy': 'Emotions', 'sadness': 'Emotions', 'anger': 'Emotions', 'fear': 'Emotions', 'surprise': 'Emotions',
    'doctor': 'Professions', 'teacher': 'Professions', 'engineer': 'Professions', 'artist': 'Professions', 'scientist': 'Profess
    'mountain': 'Nature/Landscapes', 'river': 'Nature/Landscapes', 'forest': 'Nature/Landscapes', 'ocean': 'Nature/Landscapes',
}

# Get unique categories for plotting
categories = sorted(list(set(word_categories.values())))

# Create a color map for categories
colors = plt.colormaps.get_cmap('tab10') # Corrected: Removed len(categories) as second argument
category_to_color = {category: colors(i) for i, category in enumerate(categories)}

# 2. Create a scatter plot of the 2D word embeddings
plt.figure(figsize=(15, 10))
```

```
# Loop through each word and its embedding to plot and annotate
for i, word in enumerate(valid_words):
    x, y = tsne_2d_embeddings[i, 0], tsne_2d_embeddings[i, 1]
    category = word_categories.get(word, 'Unknown') # Handle cases where a word might not be in the map
    color = category_to_color.get(category, 'gray')

    # Plot the point
    plt.scatter(x, y, color=color, s=100, label=category if word == next(iter(word_categories)) or category not in plt.gca().get_

    # 3. Annotate each data point with its corresponding word label
    plt.annotate(word, (x, y), textcoords="offset points", xytext=(5,5), ha='center', fontsize=9)

# Create custom legend handles to avoid duplicate labels for categories
handles = []
labels = []
for category in categories:
    handles.append(plt.Line2D([0], [0], marker='o', color='w', label=category,
                              markerfacecolor=category_to_color[category], markersize=10))
    labels.append(category)

# 4. Add a title to the plot
plt.title('2D t-SNE Visualization of Word Embeddings', fontsize=16)

# 5. Add labels for the x-axis and y-axis
plt.xlabel('t-SNE Dimension 1', fontsize=12)
plt.ylabel('t-SNE Dimension 2', fontsize=12)

# 6. Include a legend to distinguish between the different semantic categories
plt.legend(handles=handles, labels=labels, title='Categories', bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0.)

# Adjust plot layout to make space for the legend
plt.tight_layout(rect=[0, 0, 0.85, 1])

# 7. Display the plot
plt.grid(True)
plt.show()

print("t-SNE visualization plot generated successfully.")
```


2D t-SNE Visualization of Word Embeddings
desert
mountain
forest
river
ocean
sadness
joy
200
Categories
Animals
Cities
Devices
Emotions
Nature/Landscapes
Professions